MEDHAVA

Current state audit

What is actually in this repository, counted at the moment this file was generated.

This document exists because the same failure happened twice in one session: a figure was reported from memory and was wrong, and nothing could contradict it. A claim that there was no continuous integration, while `.github/workflows/ci.yml` sat in the tree. An archive reported at 540 files and 24.6 MB when it held 439 and 15.6 MB. So no count below is written down as a literal anywhere — each is taken from the files themselves every time this document is rebuilt.

The repository, counted

Files tracked by git	807
Lines of product code (`medhava/`)	3,164
Lines of shared core (`core/`)	4,657
Lines of registers and generators (`brand/site/`)	16,587
Tables in the production schema	151
Row-level security policies in it	2
Test files	11
Gates that can fail the build	14
Document and register generators	22

Code volume is not on this list as an achievement. The maturity level in `brand/site/audit.js` says so explicitly: a rewrite halving the line count would change nothing about what the product can do.

The design, counted

Modules	22
Apps across them	113
Business rules written	293
Rules proven by a test that runs	89
Stack layers, each with alternatives	19
Capability comparisons the owner asked for	56

What is standing up

Rung	Rows
NOT STARTED	6
SPECIFIED	98
IMPLEMENTED	16
TESTED	11
BLOCKED	1

Of 113 apps, 4 have a recorded passing test and 15 run without one. 94 are written down and not standing up. The full table, one row per app and per capability, is `REQUIREMENTS_REGISTRY.md`.

The score, and the maturity level

Score	Meaning	Rows
0	absent	7
1	concept or specification only	98
2	partial implementation	0
3	implemented but weakly verified	16
4	tested and verified	11
5	production-grade	0
	mean	**1.4 / 5**

Each score is a translation of a rung that a gate already checks, and the translation itself is bounded — a rung that demands no file on disk cannot score above 1, and only a deployed one may reach 5. Raising a score therefore requires earning the rung, which requires a command that really ran.

Maturity: level 3 — Prototype.

Two slices — stock movement and a posted sale — run on the real database inside row-level security, with tests in the gated suite and recorded passing runs. Sixteen more open in a browser over an in-page store. Everything else is a specification. Individually, those two slices have reached level 5; the product has not, because a product's level is not the best thing in it.

What would make it the next level: Level 4, Functional, would mean a business could run a real day of its work end to end in this system. It cannot: nothing purchases, nothing manufactures, nothing pays anybody, and nothing closes a period. The nearest honest test is one working day — buy something, receive it, make something from it, sell it, ship it, and see all four in the ledger — with every step on the real database. Not one of those five steps exists today beyond the selling.

What does not decide it: Not by code volume. 3,152 lines of product code is neither evidence for this level nor against it, and a rewrite that halved it would change nothing here.

What has actually been run

Command	Exit	Recorded as
`python3 engine/tests/selftest.py`	0	V-ENGINE
`npm run medhava`	0	V-MEDHAVA
`node core/tests/live.test.js`	0	V-CORE
`npm test`	1	V-FULL
`npm test`	0	V-FULL2
`npm run test:product`	0	V-PRODUCT
`npm run selftest`	0	V-SELFTEST
`node core/tests/schema.test.js`	0	V-SCHEMA
`node core/tests/packs.test.js`	0	V-PACKS
`node core/tests/core.test.js`	0	V-GROUP
`node tools/evidence.test.js`	0	V-EVTEST
`node brand/site/checkcoverage.js`	0	V-COVERAGE
`node brand/site/checkregistry.js`	0	V-REGISTRY
`node brand/site/checkregistry.js`	0	V-TRAP
`node brand/site/checkzoho.js`	0	V-ZOHO
`node brand/site/checkaudit.js`	0	V-AUDIT
`node brand/site/checkconflicts.js`	0	V-CONFLICTS
`node brand/delivery/website/mkstarter.js --verify --both`	0	V-ARCHIVE

Each was run through `tools/evidence.js`, which records the exit code the process returned, the commit, whether the tree was dirty, and the SHA-256 of the files the run was about. A non-zero entry is left in the log: deleting it would remove the only record that the failure ever happened.

What this audit cannot tell you

Stated here rather than left for a reader to discover:

- **Nothing about speed.** Every test runs on a handful of rows. No load test exists, no query plan has been reviewed, and 151 tables with row-level security on all of them is exactly the shape where a missing index

stays invisible until it is not.

- **Nothing about production.** Nothing has ever been installed anywhere. The runbook and the systemd unit are written and have never been followed by anybody.
- **Nothing about a live integration.** Every marketplace, courier, tax portal, bank and payment provider needs credentials this repository must never hold.
- **Nothing about how it compares in depth** to the products it is benchmarked against, because none of those pages could be read from here.

Where every number here comes from

Nothing in this document is typed. It is generated by `node brand/delivery/website/mkaudit.js` and every figure is read at that moment from the register that owns it:

Fact	Register	Gate
modules and apps	`brand/site/modules.js`	`checkneutral.js`, `checkshape.js`
what each has reached	`brand/site/registry.js`	`checkregistry.js`
the 0–5 score and the queue	`brand/site/audit.js`	`checkaudit.js`
the capability comparison	`brand/site/zoho.js`	`checkzoho.js`
rules and their proofs	`brand/site/rules.js`	`checkrules.js`
recorded runs	`docs/verification/EVIDENCE.md`	`tools/evidence.js --check`

A figure in this document that disagrees with its register means the document is stale. Regenerate it; `npm test` refuses a stale one.

Every technical word above, in plain language

9 words. Every technical term this document uses, in plain language, with an everyday comparison. Nothing here assumes you already know any of them.

module

One area of work in the system — sales, purchase, staff, accounts. Each is a set of screens that belong together.

Dukaan ke alag-alag counters. Ek counter bikri ka, ek kharidi ka, ek hisaab-kitaab ka.

database

Where all the information is kept, arranged so any of it can be found instantly and nothing gets lost.

Ek badi almari jisme har cheez apne fix khaane mein rakhi hai — dhoondhne ke liye poori almari palatni nahin padti.

table

One kind of information inside the database — all your customers in one, all your orders in another.

Almari ka ek khaana. Ek khaane mein sirf customers, doosre mein sirf orders.

row

One single record — one customer, one order, one payment.

Register mein ek line. Ek line matlab ek entry.

schema

The written plan of what information the system keeps and how the pieces connect.

Makaan ka naksha. Deewar uthane se pehle kaagaz pe tay hota hai kaunsa kamra kahaan hai.

row-level security

A lock inside the database itself, so one business physically cannot read another business's records — even if the software above it has a bug.

Taala darwaze pe nahin, tijori pe. Guard so bhi jaaye toh bhi tijori band rehti hai.

queue

A waiting line for work that does not have to finish this second — sending a hundred messages, building a big report.

Darzi ki dukaan ka parchi system. Kaam parchi pe likh ke lag gaya line mein; customer khada intezaar nahin karta.

continuous integration

A robot that checks every change automatically, before anyone can put it live.

Quality-check wala banda gate pe khada. Har maal nikalne se pehle usse guzarta hai.

provider

A company whose service the system uses — for messages, for payments, for artificial intelligence, for delivery.

Supplier. Ek supplier maal na de toh doosre se le lo — kaam nahin rukna chahiye.